

```

buf[100] = 'D';                               #include <stdlib.h>

putchar (buf[100]);                           #include <unistd.h>

buf[-1]='A';                                  int main( void )

putchar(buf[-1]);                             {

free(buf);                                    char* buf1 = malloc(10);

}                                              int*  buf2 = malloc(sizeof(int));

                                              write( 1 /* stdout */, buf1, 20 );

                                              exit(buf2[0]);

                                              }

```

In the above code snippet, the memory allocated through *malloc* is 100 bytes, which is addressable as *buf[0]* to *buf[99]*. However the code tries to write to *buf[100]* at Line 4, which is an out-of-bound memory write. Similarly at Line 5, it tries to read from *buf[100]*, which is an out-of-bound memory read. These two errors are typically known as ‘buffer overrun’ errors, since we are accessing beyond the upper bounds of the dynamically allocated buffer ‘*buf*’. At Line 6, the code writes to *buf[-1]* and at Line 7, it reads from *buf[-1]*. These are examples of buffer under-run issues, where the program accesses memory below the lower bound of the dynamically allocated buffer. Over-runs and under-runs typically occur due to incorrect loop index calculations, or due to the programmer writing a smaller size than actually required by the intended usage of the dynamically allocated memory.

Out-of-bound memory writes are dangerous, as they can silently corrupt other data by over-writing it. Typically, the memory allocator library stores meta-data information for each dynamically allocated block of memory just before, or after, the memory chunk allocated to the application. This meta-data is used by the dynamic memory allocator to keep track of the size of the block allocated, etc. In the above example, the invalid memory writes beyond the boundary of the ‘*malloced*’ block, which can corrupt the *malloc* header information, thereby causing silent corruption to the dynamic memory allocator meta-data. Therefore, detecting out-of-bound memory accesses is an important requirement of memory error detection tools.

What are the issues with the above code snippet? In Line 8, the dynamically allocated buffer ‘*buf1*’ is passed as an argument to the ‘*write*’ function, which tries to write 20 bytes from ‘*buf1*’ to ‘*stdout*’ (1 stands for the file descriptor *stdout*). Note that ‘*buf1*’ was simply allocated through the call to ‘*malloc*’, and was not initialised with any value. Therefore, the above code snippet tries to write uninitialised data from ‘*buf1*’ to ‘*stdout*’, which is a software bug. Next, look at line 9 where ‘*exit*’ is called with the argument ‘*buf2[0]*’. Recall that the ‘*exit*’ function takes as the first parameter the status of the exiting program. Here, it is being passed ‘*buf2[0]*’, which is uninitialised. This is another example of the use of uninitialised values in the application, again leading to a coding bug. In both the above cases, the arguments ‘*buf1*’ and ‘*buf2*’ are not initialised/defined by the application code after being dynamically allocated by the ‘*malloc*’ function call.

What is shadow memory?

While it is important that such coding bugs need to be caught by memory error checking tools, it is not easy. Many runtime memory error checking tools simply intercept *malloc/free* calls made by the application to the memory allocator library. This information is not sufficient to detect the use of uninitialised/undefined dynamic

memory locations. In order to detect such uninitialised use of dynamically allocated memory, the tool needs to track every instruction before the use instruction, to see if any of the instructions initialise that memory chunk. This requires that the tool should maintain certain information/ meta-data about every dynamic memory location, such as whether the location is defined or undefined; is it still valid or has it been freed already, etc. Such meta-data, which tracks the state of each application memory location, is known as shadow memory. For every application memory location, there is a corresponding shadow memory location that contains information on it. This is known as memory shadowing, and the memory used to hold the meta-data information is known as shadow memory. Tools that use the information in shadow memory to detect programming errors are known as shadow value tools.

Maintaining and updating information in shadow memory is expensive. It requires comprehensive instrumentation, which tracks every instruction in the application that accesses memory. Memory shadowing also incurs considerable space overheads, since a shadow memory location is used to track every application memory allocation. The tracking granularity can be different for different purposes. Consider a tool that shadows application memory at the granularity of a word. Basically, every word of application memory has meta-data associated with it, which says whether the memory word is initialised or uninitialised. Such a tool cannot detect uninitialised memory locations below the granularity of a word. Consider the following code snippet:

```

void set_one_bit_in_word(int* my_word, int
pos)

{

    my_word[pos/32] |= (1 << (pos%32));

}

int get_one_bit_in_word(int* my_word, int
pos)

```